A Toy Model of Mechanistic (Un)Faithfulness

An informal note on mechanistic faithfulness and unfaithfulness by Chris Olah. Published August 7th, 2025.

Mechanistic faithfulness is the concern that when we replace model components with sparse approximations (such transcoders), those replacements might implement different mechanisms than the original model. Every computer science student knows that there are many different algorithms which can achieve the same effect – sorting algorithms are a classic example of this! Why should we believe our transcoders learn the same mechanisms as the original underlying model?

The goal of this update is to describe mechanistic faithfulness, and to concretely illustrate it in a toy model. Althoug we've discussed this briefly in recent papers, we felt it deserved more depth. We'll also briefly explore the idea that "Jacobian matching" might help with this. Finally, we'll conclude with a discussion of how mechanistic faithfulness relates to the broader universe of concerns around SAEs and transcoders.

This note is very preliminary. We're sharing because it might be of interest to researchers working actively in this space, and we believe there's value in sharing thoughts earlier. We'd ask you to treat these results like those of a colleague sharing some thoughts or preliminary experiments for a few minutes at a lab meeting, rather than a mature paper. It is not intended for a broad audience. All claims should be taken with significant caveats and low confidence throughout.

Background: Sleepwalking into Mechanistic Faithfulness

In 2024, a number of researchers introduced what we now call transcoders : SAEs that read in a layer's input and predict its *output*. These transcoders greatly simplified circuit analysis – it was now possible to think about features having linear interactions with each other, rather than having to make unprincipled approximations through non-linearities or end up in "attribution hell".

At the time, I think we underappreciated what a radical change this was. Certainly, I underappreciated this. We were switching from using SAEs to model ***representation*** to modeling ***computation***.

When one models representation, much can be forgiven. Features are just a basis for the vector space of activations and the primary concern is whether they make the activations understandable. As long as your features are monosemantic and explain the activations, it's a valid way to understand what the activations contain. Of course, it might prove hard to understand the model's computation in that basis, but you won't be misled.

However, when one models computation, it becomes possible to have a setup that is monosemantic, and has low MSE on distribution, but achieves this through *a different computational mechanism*. This is particularly concerning since the alternate mechanisms may generalize differently off distribution. This attacks one of the deepest hopes fo mech interp: explanations we can really deeply trust.

This lack of mechanistic faithfulness is somewhat bounded by the fact that the mechanism can only diverge so far, since we're only replacing the model "one step at a time" – that is, every transcoder step predicts the ground-truth residual stream, after a single transcoder non-linearity, greatly limiting the space of mechanistic divergences. (If you force sorting algorithms to match their memory state after every computational step, that likely forces them to be t same!)

But for transcoders, we can clearly construct examples where they can *and do* mechanistically diverge despite this.

The Absolute Value Transcoder Problem and Repeated Data

Our goal is to construct a toy model illustrating mechanistic (un)faithfulness. This means we need to have a toy model which studies computation, rather than representation, such that the computation can potentially be unfaithful!

One such toy problem is the absolute value problem. We can have a one layer model try to map $x \to y = \mathrm{abs}(x)$. This can be done easily with two ReLU neurons like so:

$$y_i = \mathrm{ReLU}(x_i) + \mathrm{ReLU}(-x_i)$$

This is *exactly* identical to $\mathrm{abs}(x)$, and in fact, we can see $\mathrm{abs}(x)$ as a convenient way of writing down this exact ReLU network.

One question we could ask is "if we don't give a model enough neurons to implement this solution, how does it do computation in superposition"? We're going to leave that question to another day, and instead ask "under what circumstances can a transcoder recover these features, given a perfect model (i.e. $y = \mathrm{abs}(x)$ or equivalently $y_i = \mathrm{ReLU}(x_i) + \mathrm{ReLU}(-x_i)$) .

It turns out that our transcoders easily discover this solution by default.

(One important detail is that we're going to use tanh L1 regularization, in order to achieve closer results to L0 regularization. If we use standard L1 regularization, our transcoders essentially learn duplicate features with extra capacity, and are generally messier.)

ADDING A REPEATED DATA POINT

In the basic problem setup, we'll use the following data distribution:

$$\begin{aligned} x_i &= \mathrm{Uniform}(-1,1) \quad \text{if } \mathrm{Uniform}(0,1) \ < \ \mathrm{density} \\ x_i &= 0 \qquad\qquad\qquad\quad \text{otherwise} \end{aligned}$$

We're now going to add a special "memorization data point" to the mix. We'll define a special data point p which is on the first 3 dimensions and 0 on the others:

$$p = [1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, \ldots]$$

And then have x be p some fraction `repeat_frac` of the time.

DATAPOINT FEATURES EMERGE

If we repeat a data point like this, we begin to see a transcoder "memorization feature" circuit :

$$y += p \cdot \mathrm{ReLU}(\lambda p \cdot x - b)$$

Notice how this feature specifically activates on p and then produces p!

We'll call this a "datapoint feature".

Datapoint features like this are fine if they reflect the underlying model. Models very likely do memorize some datapoints, in which case we want features like this. But in this case, the model $x \to \mathrm{abs}(x)$ has no special notion o p. This means that these features **are not mechanistically faithful!** The transcoder is memorizing something which the underlying model didn't.

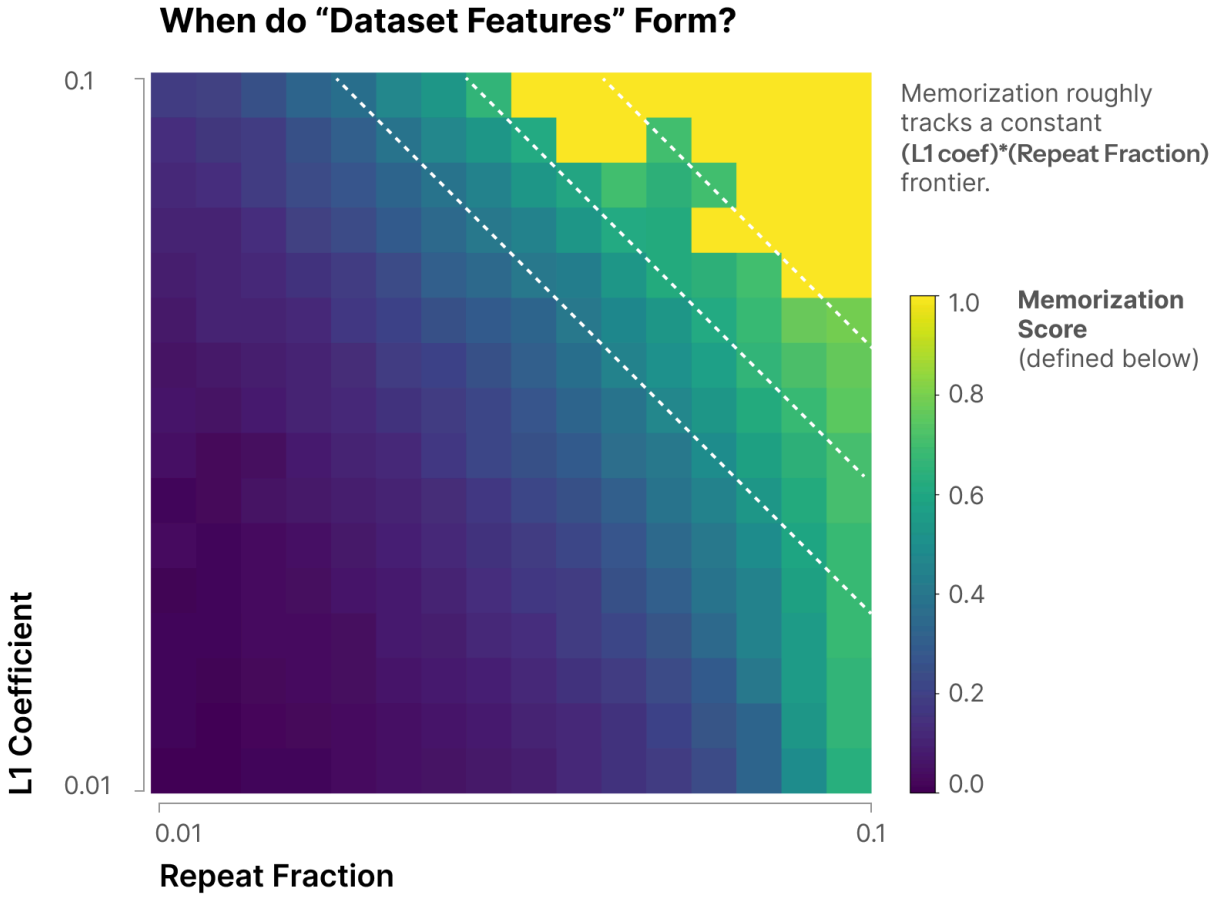
We now have an example of mechanistic unfaithfulness to explore. In the following sections, we'll investigate when these datapoint features form, and how we can get rid of them, with the goal of more generally resolving mechanistic faithfulness.

WHEN DO DATAPOINT FEATURES FORM?

Before we run more experiments, it is useful to understand exactly when we should expect these datapoint features to form in a transcoder, so that we can calibrate our experiments. When they form from the transcoder memorizing datapoint (rather than the underlying model doing so and the transcoder faithfully mimicking it) there are two necessary conditions:

1. **There must be a repeated datapoint in the transcoder training set.** More repetition makes datapoint features more likely.
2. **We must be regularizing for activations to be sparse.** This makes memorizing the data point helpful, sinc it allows us to handle that particular input more sparsely.

If we do a scan over these two hyperparameters and measure the presence of datapoint features, we see that in fac it's the product of these two properties that controls whether datapoint features form (our measure of memorizatio is defined in the next section):



ASIDE: HOW DO WE DETECT A DATAPOINT FEATURE?

For any feature f_i we can measure its contribution to y. We'll denote this as:

$$y|_{f_i}(x) = W^i_{\text{down}} f_i(x)$$

We'll define the memorization of a feature f_i on the datapoint p as the "excess" explanation of p above explaining individual three constituent dimensions we'd expect from the generalization solutions:

$$m(f_i, p) = \text{ReLU}(y|_{f_i}(p) \cdot \hat{p} - \max_{j<3} y|_{f_i}(p) \cdot e_j$$

where p is the repeated data point defined above and e_j denotes the jth unit vector. Then we define the total memorization of a model as the sum of memorization over features. This is a crude way to define memorization, bu a useful starting point.

Investigating Datapoint Features

In this section, we'll investigate our toy model in two regimes, one without repeated data points, and one with them

Our goal will be to mechanistically understand the resulting models, and directly observe the induction of data poir features.

To aid us in this, we'll introduce a new visualization which will allow us to see how each transcoder feature maps to the features we think it might be natural to learn ($\mathrm{ReLU}(x_i)$, $\mathrm{ReLU}(-x_i)$, and memorization). The mathematics c this is a little subtle, so we'll look at one example with this visualization before explaining it. This will make the mathematics more motivated and a little easier to follow.

NO REPEAT BASELINE

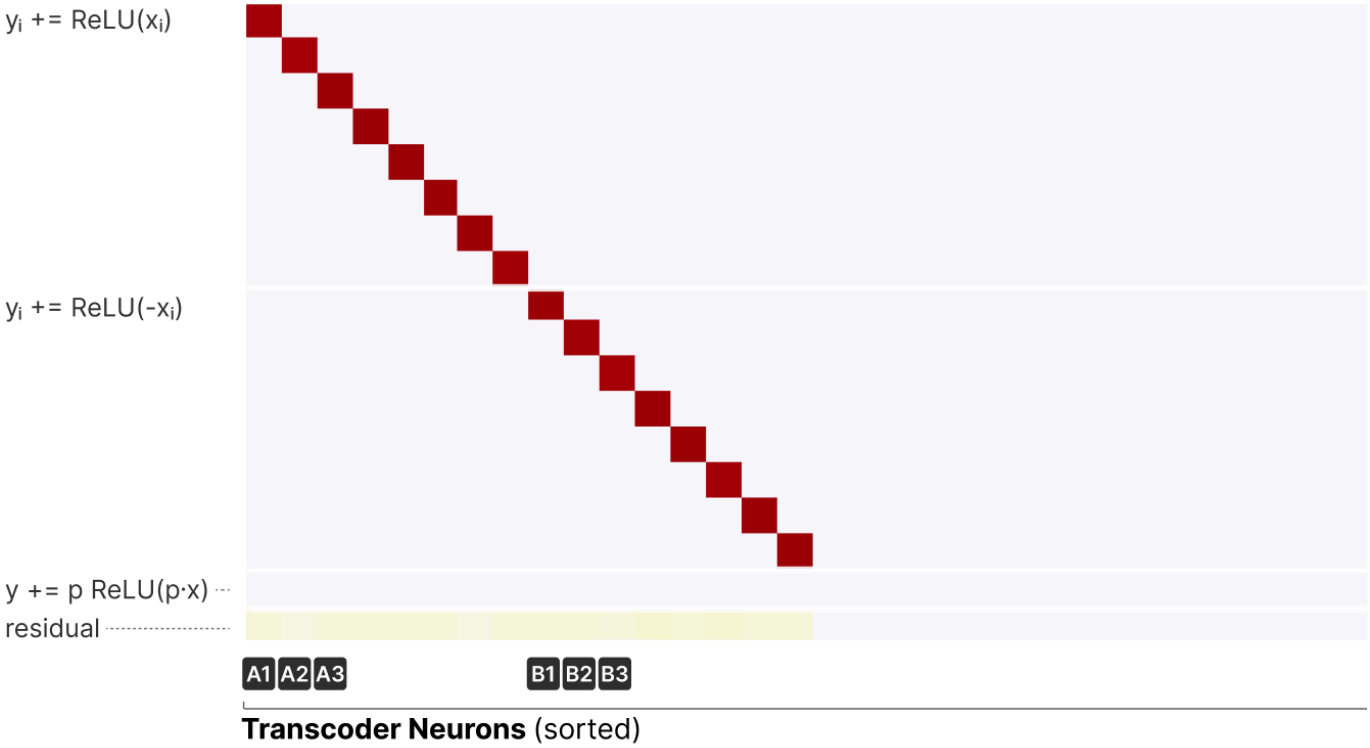
Below, you can see this visualization applied to a baseline setup, where we train a transcoder on the absolute value problem without any repetition.

This visualization is explained in detail below, but note for now that it shows a map between transcoder neurons, ar the features we might expect to learn. On the x-axis we have transcoder neurons, and on the y-axis potential features/circuits. Without repetition, we see that the transcoder learns almost exactly the expected features $y_i += \mathrm{ReLU}(x_i)$ or $y_i += \mathrm{ReLU}(-x_i)$, with a small residual. To make this even more concrete, the actual neuron weights are shown for selected neurons below.

Baseline

Circuit Type

Extent Each Transcoder Neuron Implements Ideal Circuit



Weights of Selected Transcoder Neurons

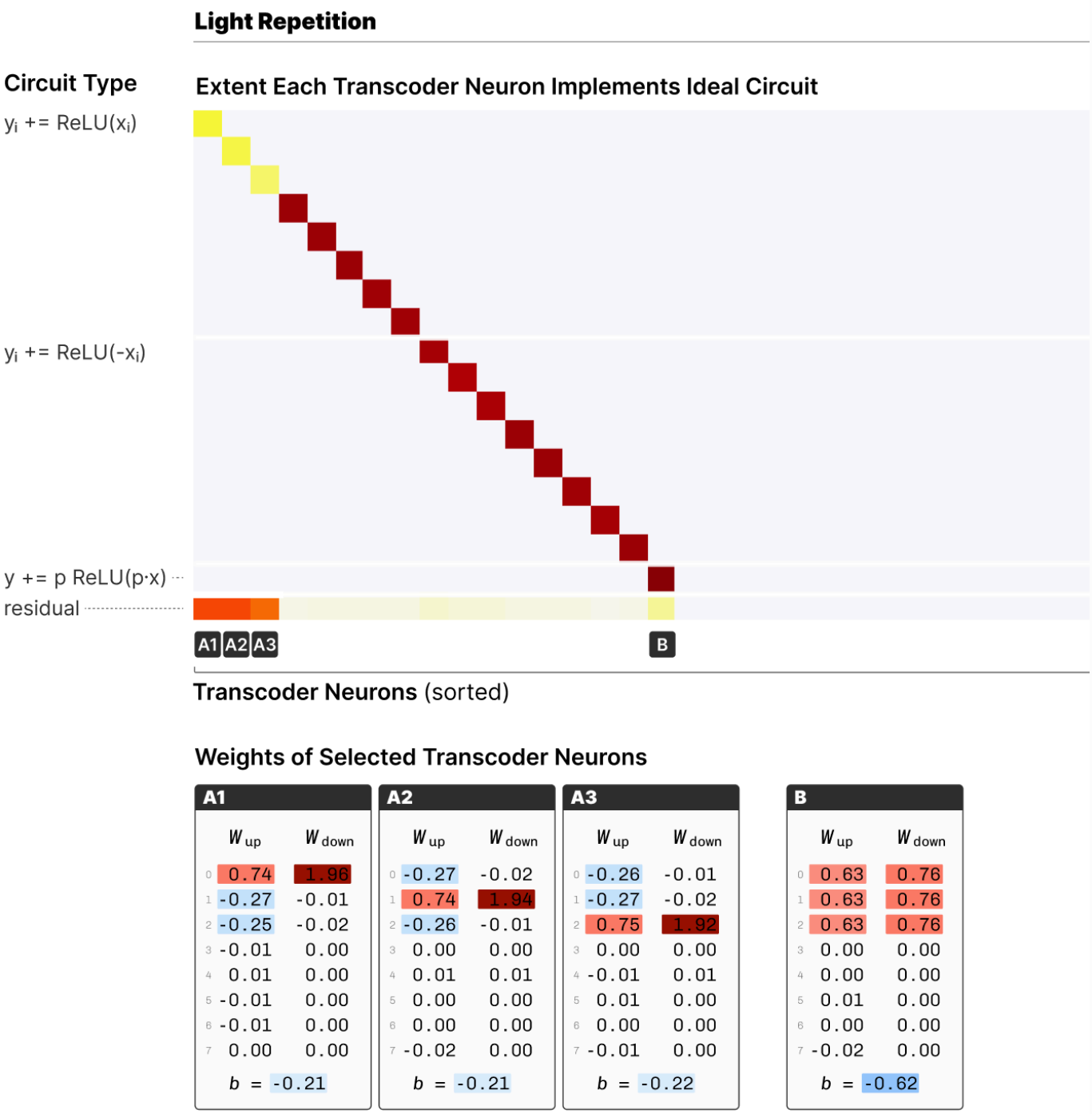
	A1		A2		A3		B1		B2		B3	
	W_{up}	W_{down}	W_{up}	W_{down}	W_{up}	W_{down}	W_{up}	W_{down}	W_{up}	W_{down}	W	
0	0.85	1.19	0.00	0.00	0.00	0.00	-0.84	1.20	0.00	0.00	0.	
1	0.00	0.00	0.88	1.16	0.00	0.00	0.00	0.00	-0.83	1.22	0.	
2	0.00	0.00	0.00	0.00	0.78	1.30	0.00	0.00	0.00	0.00	-0.	
3	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.	
4	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01	0.00	0.00	0.	
5	-0.01	0.00	0.01	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.	
6	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	-0.01	0.00	-0.	
7	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.	
	b = -0.05		b = -0.05		b = -0.05		b = -0.05		b = -0.05		b	

ASIDE: HOW DOES THIS VISUALIZATION WORK?

The key idea is that we have a "library" of ideal circuits we think the transcoder might learn (the regular ReLU ones and memorization). We consider the "contribution" $y|_{f_i}$ each ideal circuit would produce (as defined above). We consider a vector of such contributions over a large sample of data points. This produces a very high-dimensional vector. We can construct a similar vector for each of our learned transcoder features; these vectors are constructed over the same data points so that the vectors are comparable. We then do compressed sensing to try to explain the observed contributions of each transcoder feature circuit in terms of the ideal circuits. At the end, we get a matrix o transcoder features by ideal features, and also a residual that can't be explained.

LIGHT REPETITION

We can now ask what happens if we introduce a repeated data point! (We only use a relatively small amount of repeated data for this setup, 5%, such that memorization is marginal).



We can now see a data point feature (see bottom right), as expected. We also have features corresponding to the three individual features which are used in the repeated data point, but they are rotated so that they don't activate on the repeated data point.

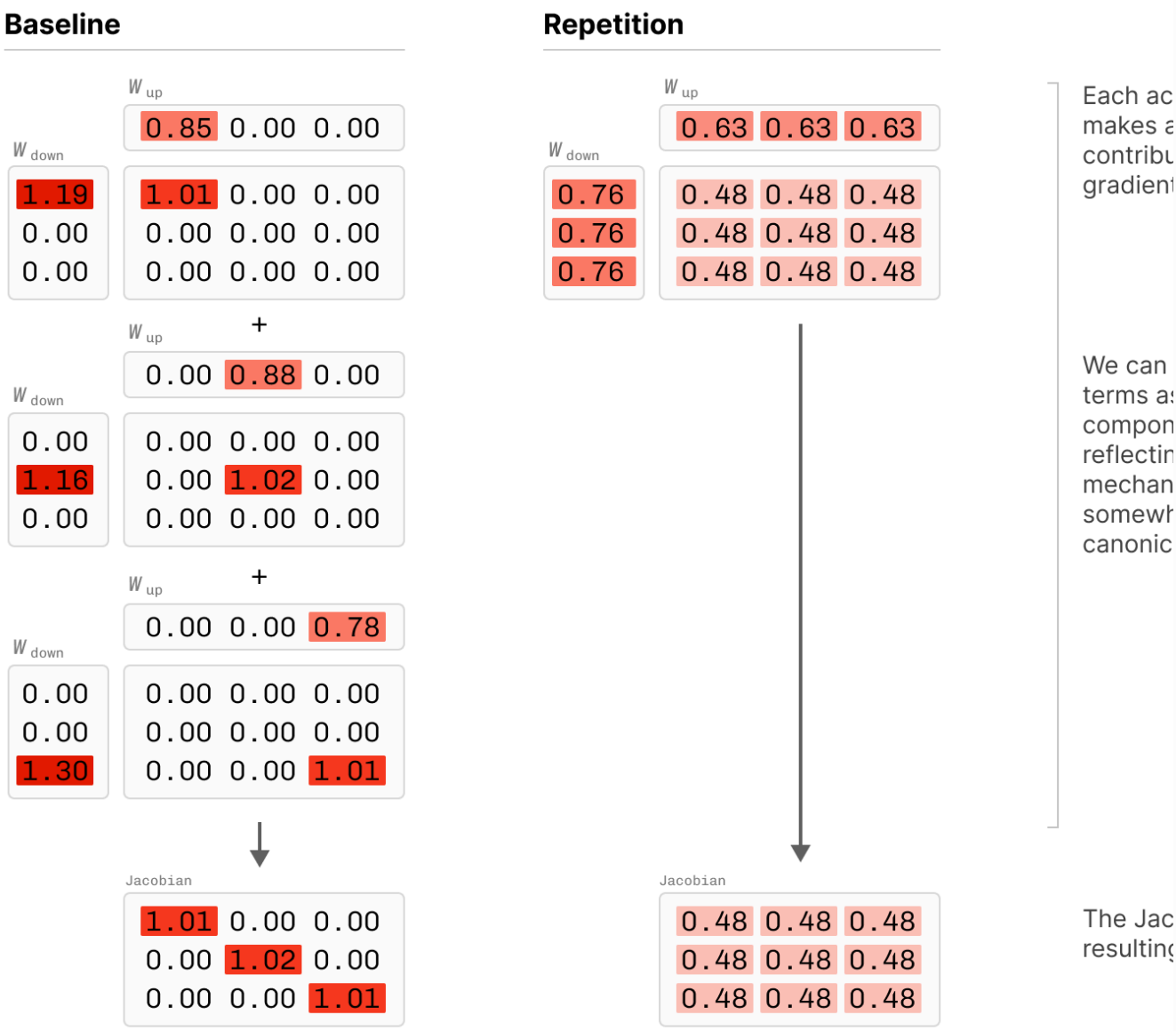
This demonstrates that mechanistic faithfulness can be a real problem, even in very simple setups!

Jacobian Regularization: A Potential Solution?

Mechanistic faithfulness is a serious problem, but we don't think the issue is hopeless. In this section, we propose a

connection between mechanistic faithfulness and a layer's Jacobian, and then demonstrate that we can exploit this t solve mechanistic faithfulness, at least in some toy problems. (We don't claim this is the ideal solution, and indeed we'll note some issues with it. Our goal is simply to show that there are productive paths forward to explore.)

The connection to Jacobian's may not be obvious at first. One way to see it is to consider how the features learned transcoders in our previous two examples produce very different Jacobians on the memorized data point:



In the baseline case, we see a diagonal matrix (telling us the features are independent). But in the repeated data cas with the datapoint feature, we see that the "reason" each feature is active locally is due to all the other features (which should be irrelevant!).

There's a deeper technical reason this is principled. For a single layer, there's a very important way in which the Jacobian (as a function across data points) hits upon the nature of its mechanisms. Intuitively, if you want to understand the mechanisms, you should look at the weights – this is the motivation behind circuits work, and also behind recent "weight-based interpretability" (e.g.). But the weights of neural networks aren't canonical – there's kinds of ways you can transform them, such as permuting neurons, or inversely scaling the input and output weight They also need context about how input statistics will cause non-linearities to behave. For a ReLU layer, we can understand its Jacobian as being a sum of outer products corresponding to its input and output weights. In some sense, this is a natural canonicalized form of the parameters, and also attaches (when considered as a function over data points) information about when neurons co-activated.

JACOBIAN MATCHING

Given this observation, a natural response is what we'll call "Jacobian matching". We penalize the difference betwee our transcoder's jacobian and the true jacobian:

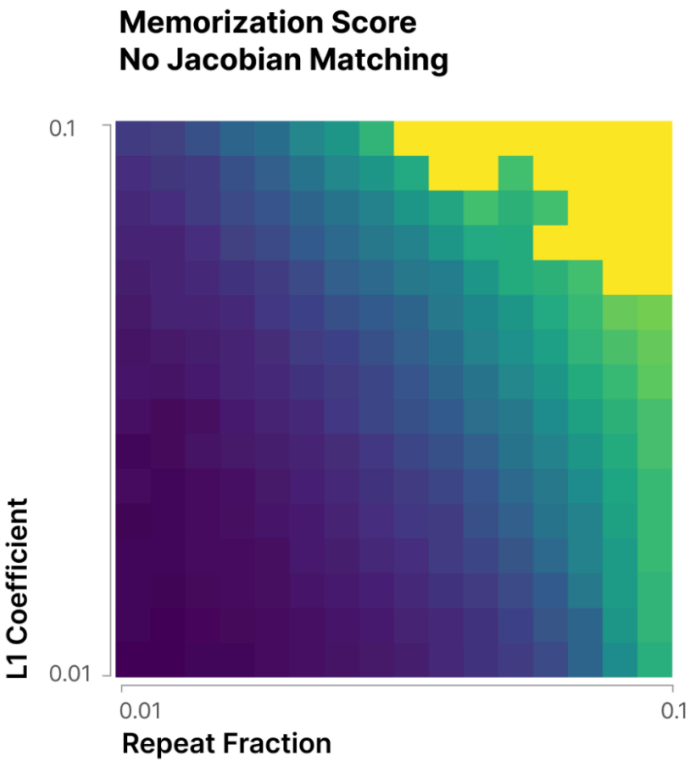
$$||J_{\text{MLP}} - J_{\text{Transcoder}}||_F^2$$

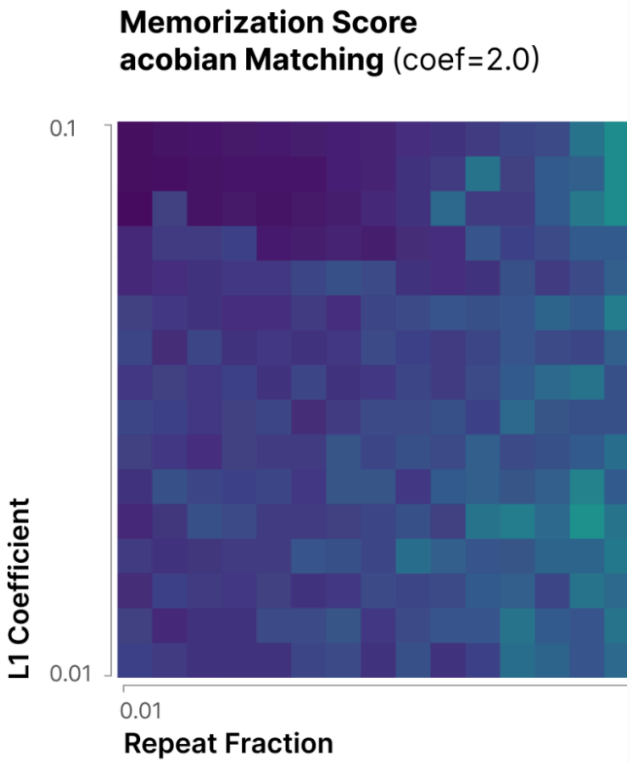
In practice, this objective would be extremely expensive. Instead, we exploit that this is equivalent in expectation to:

$$E_v K||J_{\text{MLP}}v - J_{\text{Transcoder}}v||_2^2$$

where v is a random vector. This can be computed cheaply with backprop.

It turns out that adding this to our loss *dramatically* helps our datapoint feature memorization metric.





JACOBIAN MATCHING CAN SOLVE MECHANISTIC FAITHFULNESS (IN THIS EXAMPLE)

Let's return to our repeated data point example, which produced a data point feature above. What happens if we regularize it to match the ground truth Jacobian? It turns out to basically fix the problem!

Repetition + Jacobian Matching

Circuit Type

y_i += ReLU(x_i)

y_i += ReLU(-x_i)

y += p ReLU(p·x)

residual

Extent Each Transcoder Neuron Implements Ideal Circuit

Transcoder Neurons (sorted)

Weights of Selected Transcoder Neurons

A1

	W_{up}	W_{down}
0	1.02	0.96
1	-0.04	0.07
2	-0.04	0.07
3	0.00	0.00
4	0.00	0.00
5	0.00	0.00
6	0.00	0.00
7	0.00	0.00

b = -0.06

A2

	W_{up}	W_{down}
0	-0.04	0.07
1	1.01	0.97
2	-0.04	0.07
3	0.00	0.00
4	0.00	0.00
5	0.00	-0.01
6	0.00	0.00
7	0.00	0.00

b = -0.06

A3

	W_{up}	W_{down}
0	-0.04	0.07
1	-0.04	0.07
2	0.99	0.99
3	0.01	0.00
4	0.00	-0.01
5	0.00	0.00
6	0.00	0.00
7	0.00	0.00

b = -0.06

We see that this gets rid of the memorization feature, but that our transcoder features for the features used on the repeated point are a bit rotated (see residual at bottom left).

CAN THE TRANSCODER CHEAT?

In the adversarial attack robustness literature, there's a problem called "gradient masking", where models will learn manipulate their gradients. The general trick we observe is that they can create features with large weights and thus large gradients, but very negative biases so that they're barely active and don't contribute much to the L1 objective

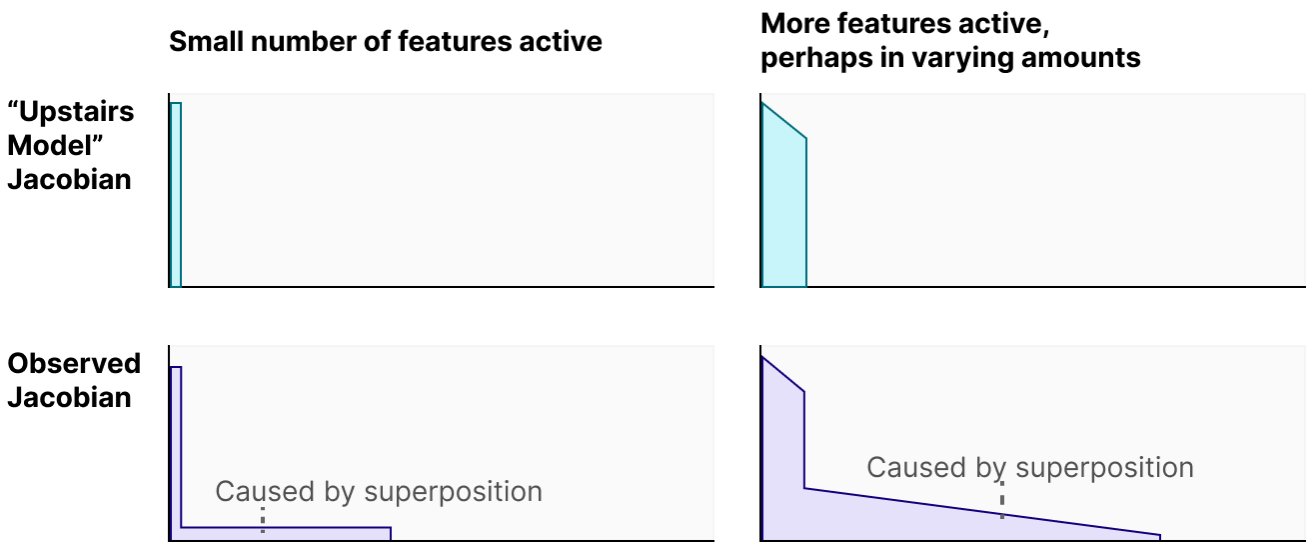
Note that this is exploiting a gap between our L1 objective, and the L0 objective we wish to optimize. Using tanh L1 helps significantly here, since it moves us closer to L0.

In principle, it seems like there are other optimization terms we could add to push back on this if it was an issue. Fo example, it might be interesting to penalize the L1 of different transcoder features contributions to the Jacobian, to penalize attempts to manipulate the gradient like this. This can be cheaply approximated, since it's just the norms o the input and output weights multiplied together, masked by the Jacobian of the transcoder activation function.

SUPERPOSITION AND JACOBIAN MATCHING

Surprisingly, we may not want to perfectly match the Jacobian. This is because we believe the models we study are superposition, and we believe that doing superposition will "blur" the Jacobian.

One way to think about this is to ask what we expect the spectrum of the Jacobian to look like. Very naively, we mig expect something like this:



The tail is an artifact of superposition, and we shouldn't care about modeling it. Concretely, it corresponds to featur spread over multiple neurons, which also represent other features. If a single feature is active and it is spread over 1 neurons, we should expect one large singular value in the Jacobian corresponding to that feature, and 9 small singular values corresponding to other things those neurons help represent.

This intuition makes us believe that, while we want to push towards better Jacobian matching, we shouldn't expect t match it perfectly.

Conclusion

We've seen that mechanistic faithfulness can be a real concern!

In our thinking, it's become more than this. In the ongoing discourse on SAEs/transcoders and their potential weaknesses, we've found it helpful to refactor concerns into two broad categories:

1. **Mechanistic Faithfulness** - Is the transcoder capturing the true mechanisms? Failure here cuts at the heart everything we hope for in mechanistic interpretability. On the other hand, as long as we can succeed at this category, at least transcoders can be used to reliably learn true things! In practice, this is probably a spectru

2. **Simplicity.** Are transcoders capturing mechanisms in the simplest/cleanest manner? One could imagine cas where the same mechanism can be explained in multiple different ways. For example, it's possible there exis feature manifolds / multidimensional features, but that they can be accurately approximated with a large number of discrete features. Failure here is more of a tax on how hard it will be to understand things.

In practice, many concerns people raise speak to both of these categories to some extent.

Both of these are major concerns, but we're presently most concerned with mechanistic faithfulness. Although success on it is probably more of a spectrum than binary, we see it as cutting at the very heart of why we care abou mechanistic interpretability. Thankfully, it seems like there are promising paths forward.

Acknowledgments

The ideas described in this note were greatly shaped and informed by discussions with Josh Batson, Michael Sklar, Jack Lindsey, Adly Templeton, and Siddharth Mishra-Sharma. In particular, many of these individuals did very significant thinking expanding the ideas in this post, and pursuing the much more ambitious project of how they might relate to real models. The only reason they are not authors is that we scoped this post quite narrowly and didn't include various extensions, in the interests of publishing this earlier.

We're more broadly grateful for comments from the entire Anthropic interpretability team.